

中国计量大学信息工程学院人工智能系

实验报告

实验课程： 算法与数据结构 实验名称： exp4-NMS 算法

班 级： 24 智能 2 班 学 号： 2400305209

姓 名： 史航安 实验日期： 2025/12/11

一、实验目的

1. **深入理解排序算法**：通过亲手实现快速排序、归并排序、堆排序、插入排序和冒泡排序，掌握不同排序算法的核心思想、时间复杂度特征及其在不同规模数据下的表现。2. **掌握 NMS 算法原理与应用**：学习非极大值抑制（Non-Maximum Suppression, NMS）算法在计算机视觉目标检测中的作用，理解其基于置信度排序和 IoU（交并比）计算的筛选机制。3. **性能分析与优化**：探究不同排序算法对 NMS 整体性能的影响，分析数据规模（ N ）与数据分布（随机 vs 聚集）对算法效率的影响；设计并实现基于栅格分桶的优化版 NMS，对比其与基础版 NMS 的性能差异。4. **工程实践能力**：提升使用 C++ 及 STL（Standard Template Library）进行算法设计、模块化编程、性能测试及构建工具（CMake）使用的综合能力。

二、实验内容

1. 排序算法实现：

- 基于自定义 **Vector** 容器实现五种排序算法：快速排序（Quick Sort）、归并排序（Merge Sort）、堆排序（Heap Sort）、插入排序（Insertion Sort）、冒泡排序（Bubble Sort）。
- 实现比较器（Comparator），支持按边界框（Box）的置信度（Score）进行降序排列。

2. NMS 算法实现：

- **基础 NMS**：实现标准的 $O(N^2)$ NMS 流程，包括排序、IoU 计算与重叠抑制。
- **栅格 NMS**：设计基于空间网格索引（Grid-based）的优化 NMS，通过将边界框映射到二维网格减少无效的 IoU 计算，提升大规模数据下的处理效率。

3. 数据生成:

- 设计 **随机分布生成器**: 在画布上均匀随机生成边界框。
- 设计 **聚集分布生成器**: 基于多个中心点生成高斯分布的簇状边界框, 模拟真实场景中目标密集分布。
- 支持生成不同规模的数据集 ($N = 100, 1000, 3000$)。

4. 性能测试:

- 记录每种排序算法在不同 NMS 实现 (基础 vs 栅格)、不同数据分布、不同数据规模下的运行时间。
- 输出格式化的性能对比报告。

三、实验环境

- 操作系统: Windows 11
- 开发工具: Trae IDE / VS Code
- 编译器: MinGW GCC / MSVC (支持 C++17)
- 构建系统: CMake 3.16+
- 硬件环境: Intel Core i7 / 16GB RAM (示例配置)
- 核心库: MySTL (本地自研简易 STL 库, 包含 Vector 等实现)

实验成绩: _____

主管教师签名: _____

四、实验数据记录（源程序或算法设计思想）

1. 数据结构定义

实验核心数据结构为 `Box`，包含左上角坐标 (x_1, y_1) 、右下角坐标 (x_2, y_2) 、置信度 *score* 以及原始索引 *id*。

IoU 计算实现

```
1 struct Box {
2     float x1, y1, x2, y2, score;
3     int id;
4 };
5
6 // IoU calculation function
7 static inline float iou(const Box& a, const Box& b) {
8     // Calculate intersection area
9     float ix1 = std::max(a.x1, b.x1);
10    float iy1 = std::max(a.y1, b.y1);
11    float ix2 = std::min(a.x2, b.x2);
12    float iy2 = std::min(a.y2, b.y2);
13    float iw = ix2 - ix1;
14    float ih = iy2 - iy1;
15    if (iw <= 0 || ih <= 0) return 0.0f; // No intersection
16
17    float inter = iw * ih;
18    float areaA = (a.x2 - a.x1) * (a.y2 - a.y1);
19    float areaB = (b.x2 - b.x1) * (b.y2 - b.y1);
20    float uni = areaA + areaB - inter; // Union area
21
22    if (uni <= 0) return 0.0f;
23    return inter / uni;
24 }
```

2. 排序算法设计

本次实验实现了五种排序算法，通过模板函数与自定义比较器实现对 `Box` 对象的通用排序。

(1) 快速排序 (Quick Sort)

采用分治策略，选择 `pivot`（本实现选取区间末尾元素），将数组划分为两部分，左侧元素均优于 `pivot`，右侧元素均劣于 `pivot`，递归处理。

```
1 template<typename T, typename Compare>
2 static int part(Vector<T>& a, int lo, int hi, Compare comp) {
3     T pivot = a[hi - 1];
4     int i = lo;
```

```

5     for (int j = lo; j < hi - 1; ++j) {
6         if (comp(pivot, a[j])) { // Descending comparison: a[j] > pivot
7             vswap(a[i], a[j]); ++i;
8         }
9     }
10    vswap(a[i], a[hi - 1]);
11    return i;
12 }

```

(2) 冒泡排序 (Bubble Sort)

重复遍历列表，比较相邻元素，若顺序错误则交换，直到没有交换发生。本实验中为了观察 $O(N^2)$ 排序对 NMS 的影响而加入。

```

1 template<typename T, typename Compare>
2 static void sort_bubble(Vector<T>& a, Compare comp) {
3     int n = a.size();
4     bool swapped = true;
5     while (swapped) {
6         swapped = false;
7         for (int i = 1; i < n; ++i) {
8             if (comp(a[i - 1], a[i])) { // If prev < next (lower score),
swap
9                 vswap(a[i - 1], a[i]);
10                swapped = true;
11            }
12        }
13        --n;
14        if (n <= 1) break;
15    }
16 }

```

(3) 堆排序 (Heap Sort)

利用堆数据结构（本例隐含构建大顶堆），将堆顶元素（最大值）与末尾交换，并缩小堆范围进行下滤（Heapify）。由于需要降序，实际操作中逻辑稍作调整或最终反转。

(4) 归并排序 (Merge Sort)

采用分治法，将序列二分递归排序，再合并两个有序子序列。稳定性好，但需要 $O(N)$ 额外空间。

(5) 插入排序 (Insertion Sort)

构建有序序列，对于未排序数据，在已排序序列中从后向前扫描，找到相应位置并插入。小规模数据表现尚可，大规模下性能较差。

3. NMS 算法设计

(1) 基础 NMS (Basic NMS)

算法流程：

1. 按置信度对所有框进行排序（降序）。
2. 初始化所有框为“未抑制”状态。
3. 遍历排序后的列表：
 - 若当前框 A 已被抑制，跳过。
 - 保留框 A 。
 - 遍历 A 之后的所有框 B ：
 - 若 B 未被抑制且 $\text{IoU}(A, B) > \text{threshold}$ ，则抑制 B 。

时间复杂度：排序 $O(N \log N)$ + 抑制 $O(N^2)$ ，总体 $O(N^2)$ 。

(2) 栅格分桶 NMS (Grid NMS)

算法流程：

1. 按置信度排序。
2. 将图像划分为 $W \times H$ 的网格（例如 50×50 像素为一格）。
3. 预处理：将每个框根据中心点坐标放入对应的网格桶中。
4. 遍历排序后的列表：
 - 若当前框 A 已被抑制，跳过。
 - 保留框 A 。
 - 计算 A 中心所在的网格及其 3×3 邻域网格。
 - 仅遍历邻域网格中的框 B ：
 - 若 B 的索引在 A 之后且未被抑制，计算 IoU 并进行抑制。

优势：将抑制阶段的全局搜索缩小为局部搜索，大幅降低比较次数。

```
1 static Vector<Box> nms_grid(Vector<Box> boxes, float thr, SortAlgo algo,
2   float cell) {
3     apply_sort(boxes, algo);
4     // ... Calculate grid dimensions and allocate ...
5
6     // Build grid index
7     Vector<Vector<int>> grid;
```

```

7 // ... Insert each box index into corresponding grid[idx] ...
8
9 Vector<Box> keep;
10 // ... Traverse each box ...
11 // Search potential overlapping boxes ONLY in 3x3 neighbor grids
12 for (int yy = gy0; yy <= gy1; ++yy) {
13     for (int xx = gx0; xx <= gx1; ++xx) {
14         int idx = yy * gw + xx;
15         for (int t = 0; t < grid[idx].size(); ++t) {
16             // ... Compare IoU and suppress ...
17         }
18     }
19 }
20 return keep;
21 }

```

五、实验数据分析及结论（程序运行结果及分析）

1. 实验结果数据

以下数据基于 1000×1000 画布，IoU 阈值 0.5，栅格大小 50 的测试环境。单位：毫秒 (ms)。

表 1: 不同算法在随机分布下的 NMS 耗时对比

算法类型	规模 N=100	规模 N=1000	规模 N=3000	时间复杂度
基础 NMS				
快速排序	0	9	66	$O(N^2)$
归并排序	0	9	66	$O(N^2)$
堆排序	0	10	69	$O(N^2)$
插入排序	0	9	87	$O(N^2)$
冒泡排序	0	13	112	$O(N^2)$
栅格 NMS				
快速排序	0	1	4	$O(N \log N + K \cdot N)$
归并排序	1	1	5	$\approx O(N)$ (近似)
堆排序	0	1	5	
插入排序	1	3	22	
冒泡排序	1	6	43	

表 2: 不同算法在聚集分布下的 NMS 耗时对比

算法类型	规模 $N=100$	规模 $N=1000$	规模 $N=3000$	性能分析
基础 NMS				
快速排序	0	7	54	
插入排序	0	11	68	
冒泡排序	0	13	98	
栅格 NMS				
快速排序	0	2	13	局部搜索优势
插入排序	0	4	31	
冒泡排序	0	9	57	

2. 结果分析

(1) 排序算法的影响

在基础 NMS 中，整体耗时主要由 $O(N^2)$ 的抑制过程主导。

- 当 $N = 3000$ 时，快速、归并、堆排序差异不大（约 66ms），因为排序本身的 $O(N \log N)$ 耗时占比相对较小。
- 插入排序（77ms）和冒泡排序（112ms）明显更慢，尤其是冒泡排序，其 $O(N^2)$ 的排序开销与 NMS 的抑制开销叠加，导致整体性能最差。
- **结论：**对于 NMS，选择 $O(N \log N)$ 的排序算法（如快速排序）是必要的，特别是当 N 较大时。

(2) 基础 NMS vs 栅格 NMS

- **性能提升显著：**栅格 NMS 在 $N = 3000$ 时将耗时从约 66ms 降低到了 4-5ms，加速比超过 10 倍。这是因为基础 NMS 需要计算几乎所有未抑制框对的 IoU，而栅格 NMS 仅计算空间邻域内的框。
- **复杂度优化：**栅格 NMS 成功将抑制阶段的复杂度从 $O(N^2)$ 降低到了接近线性的水平（取决于数据密度和栅格大小）。
- **排序瓶颈凸显：**在栅格 NMS 中，由于抑制阶段极快，排序阶段的耗时占比大幅提升。例如，冒泡排序在栅格 NMS 下的总耗时（43ms）远高于快速排序（4ms），这清晰地展示了低效排序算法对优化后系统的拖累。

(3) 数据分布的影响

- **聚集分布：**在聚集分布下，框的重叠率更高。基础 NMS 在聚集分布下（54ms）略快于随机分布（66ms），这可能是因为高分框能更快地抑制大量重叠框，从而减少了后续遍历的次数（被抑制的框不需要再去抑制别人）。
- **栅格效率：**聚集分布下栅格 NMS 的耗时（13ms）略高于随机分布（4ms），这是因为聚集区域内的网格桶中包含更多元素，导致局部邻域内的比较次数增加。

3. 结论

1. **算法选择：**实际工程中应首选 **快速排序 + 栅格/索引优化 NMS** 的组合。
2. **优化策略：**当数据规模增长时，单纯优化排序算法收益有限（受限于 $O(N^2)$ 的抑制瓶颈）；引入空间索引（如栅格）打破 $O(N^2)$ 瓶颈是性能提升的关键。
3. **分布敏感性：**算法性能受数据分布影响，聚集型数据虽然利于基础 NMS 的“早停”，但会增加栅格 NMS 的桶内负载，需根据场景调整栅格大小。

六、实验中的问题及心得体会

1. 遇到的问题

1. **中文乱码问题：**在 Windows 控制台输出实验结果时，初期出现乱码。- **解决：**引入 `windows.h` 并在 `main` 函数中设置 `SetConsoleOutputCP(CP_UTF8)`，强制控制台使用 UTF-8 代码页。

2. **栈溢出风险：**在实现快速排序和归并排序的递归版本时，考虑到若数据量极大（如 $N > 10^5$ ），可能导致栈溢出。- **思考：**虽然本次实验 $N \leq 3000$ 未出现此问题，但生产环境应考虑改用非递归实现或 `introspective sort`。

3. **栅格边界处理：**在实现栅格 NMS 时，处理边界框中心点坐标映射到网格索引时，需小心处理数组越界（例如坐标正好等于图像宽高的情况）。- **解决：**增加了 `clamp` 逻辑，确保 `gx`, `gy` 始终在 `[0, gw-1]` 范围内。

2. 心得体会

通过本次实验，我不仅复习了经典的排序算法，更深刻体会到了 ****“算法复杂度”**** 在实际运行时间上的具体体现。- ****理论与实践结合****：看着 $O(N^2)$ 的冒泡排序在数据量增加时耗时剧增，而 $O(N \log N)$ 算法保持稳健，验证了理论分析的正确性。- ****优化的层次****：优化不能只盯着某一个环节。在 NMS 中，优化排序只能提升一部分性能，而改变算法逻辑（引入栅格索引）降低整体复杂度阶数，才能带来质的飞跃。这启示我在未来的开发中，要先分析系统的**主要瓶颈**，再对症下药。- ****工程细节****：从 CMake 构建脚本的编写到 Windows 字符集的处理，这些工程细节虽然琐碎，但却是保证程序正确运行和跨平台移植的基础。